Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Paradigmas de Programacion

Tarea #2

Racket

Estudiante	Carné
Andres Enrique Blanco Coto	2022108841
Gabriel Bolaños Barboza	2022327511
Fabiana María Moreno Castrillo	2023153700

Profesor:

Luis Alonso Barboza Artavia

9 de abril del 2026

Índice

I. Introducción 2

II. Descripción de Funciones 2
- II-A. Función mover-izquierda 2
- II-B. Función fusionar 2
- II-C. Función agregar-nueva-baldosa 2
- II-D. otras funciones 2

III. Estructuras de Datos 2

IV. Descripción de Algoritmos 3
- IV-A. Movimiento del tablero 3
- IV-B. Fusión de baldosas 3
- IV-C. Generación de nuevas baldosas 3
- IV-D. Diagramas 4

V. Problemas sin Solución 4

VI. Plan de Actividades 4

VII. Problemas Encontrados 6

VIII. Conclusiones y Recomendaciones 6
- VIII-A. Recomendaciones 6

IX. Repositorio 7

X. Manual de usuario 7
- X-A. Descripción del sistema 7
- X-B. Requisitos del sistema 7
- X-C. Ejecución del programa 7
- X-D. Uso del sistema 7
- X-E. Condiciones del juego 7
- X-F. Ejemplo de uso 8
- X-G. Consideraciones 8

Referencias 8

I. Introducción

El presente docuemnto describe el desarrollo de una aplicacion basada en un juego 2048, se implemento una solucion utilizando el lenguaje Raket, aplicando el paradigam de programacion funcional, el uso de recursividad y estructuras de datos basadas en listas. El objetivo es cumplir con los requerimientos funcionales garaqntizando una correcta representacion del tablero y la logica del juego.

II. Descripción de Funciones

El sistema se funciona a partid de dos modulos principales, estos son La: logica del juego y la interfaz grafica. La logica del juego se encarga de manipular el estado del tablero, mientras que la interfaz permite visualizarlo e interactuar con el usuario.

II-A. Función mover-izquierda

Esta es una de las funciones principales. Ya que esta porcesa cada fila del tablero desplazando los valores hacia la izquierda, eliminando ceros intermedios y aplicando la fusion de baldosas iguales. Para esto se apoya de la funcion ”fucionar”. de manera analoga, los movimiento en otras direcciones se implementan reulitizando la logica base mediante transfomaciones del tablero, como inversion de filas o transposicion de la matriz.

II-B. Función fusionar

Esta fucnion recorre la lista y combina elementos iguales consecutivos.en una sola baldosa y asi evitando multiples fusiones en una misma jugada.

II-C. Función agregar-nueva-baldosa

La funcion agregar nueva baldosa se encarga de insertar un nuevo valor en una posicion vacia aleatroria del tablero despues de cada movimeinto valido, y asi garantizando asi la continuidad del juego.

II-D. otras funciones

Ademas se implementean funciones auxiliares como gano: que venrifican si existe una baldosa con valor de 2048. O perdio: que determinan si no quedan movimientos posibles. Estas funciones permiten controlar el flujo del juego y definir las condiciones de finalización.

III. Estructuras de Datos

El tablero del juego se representa como una lista de listas de tamaño 10x8, donde cada sublista corresponde a una fila del tablero.

Ejemplo:

```
'((2 0 0 2 0 0 0 0)
  (0 4 0 0 0 0 0 0)
  (0 0 2 0 0 0 0 0)
  (0 0 0 0 0 0 0 0)
  (0 0 0 0 0 0 0 0)
  (0 0 0 0 0 0 0 0)
  (0 0 0 0 0 0 0 0)
  (0 0 0 0 0 0 0 0)
  (0 0 0 0 0 0 0 0)
  (0 0 0 0 0 0 0 0))
```

Cada elemento representa una celda del tablero, donde el valor 0 indica una celda vacía.

Se eligió esta estructura debido a su simplicidad y facilidad para recorrer filas y columnas, lo cual permite implementar de manera directa los movimientos y las fusiones del juego.

IV. Descripción de Algoritmos

IV-A. Movimiento del tablero

El movimiento del tablero se realiza aplicando una transformacion sobre cada fila o columna, dependiendo de la direccion indicada por el usuario. para el caso de movimiento hacia la izquierda, se procesan las filas eliminando los valores cero, desplazando los elementos hacia la izquierda y posteriormente aplicando la lógica de fusión. Para otras direcciones, se reutiliza este mismo procedimiento mediante transformaciones auxiliares como la inversión de filas o la transposición del tablero. Cada movimiento genera un nuevo estado sin modificar el anterior.

IV-B. Fusión de baldosas

La fusion de baldosas se inplementa mediante un reccorido recursivo de cada fila ya desplazada. Cuando dos elementos consecutivos tienen el mismo valor, se combinan en una unica baldosa cuyo valor es la suma de ambos, y se evita que esta nueva baldosa vuelva a fusionarse en la misma jugada. El argoritmo continúa procesando el resto de la lista hasta completar la fila. Este enfoque garantiza que se respeten las reglas del juego en cuanto a combinaciones.

IV-C. Generación de nuevas baldosas

Después de cada movimiento válido, se genera una nueva baldosa en una posición vacía del tablero. Para ello, primero se identifican todas las posiciones disponibles (celdas con valor cero), y luego se selecciona una de forma aleatoria. En dicha posición se inserta un valor, el cual puede ser 2 o 4, siguiendo una distribución probabilística definida. Este proceso asegura la progresión del juego y aumenta su complejidad conforme el tablero se llena.

IV-D. Diagramas

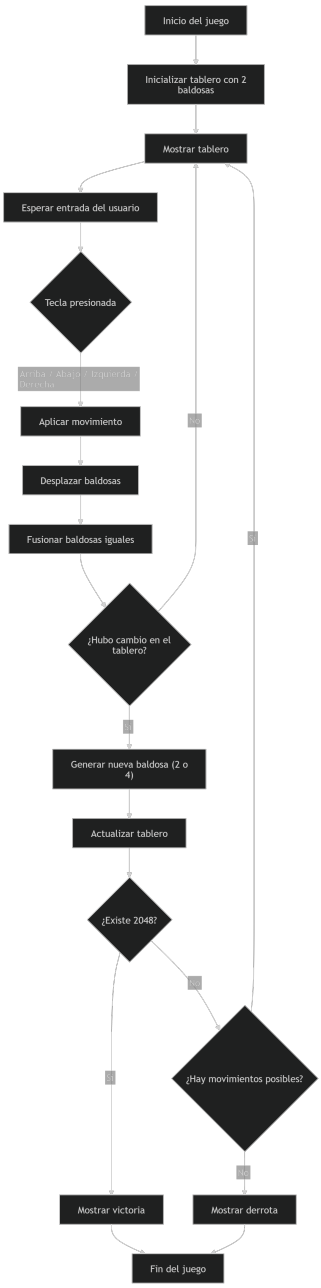

Figura 1. Caption

V. Problemas sin Solución

Durante el desarrollo del proyecto no se identificaron problemas sin solución. Todas las dificultades encontradas fueron analizadas y resueltas satisfactoriamente, permitiendo completar la implementación de los requerimientos establecidos y asegurar el correcto funcionamiento del sistema en sus distintos componentes.

VI. Plan de Actividades

1. Análisis de requisitos y alcance: (3 abril 2026)
   - Qué hacer: leer el enunciado y definir funcionalidades principales del juego 2048.
   - Debe quedar listo: lista clara de requerimientos funcionales y no funcionales.

- Errores comunes: no considerar la interfaz gráfica desde el inicio.
- Verificar antes de seguir: entender completamente las reglas del juego.

2. Diseño de estructuras de datos: (3 abril 2026)
   - Qué hacer: definir la representación del tablero como lista de listas.
   - Debe quedar listo: formato del tablero y manejo de celdas vacías.
   - Errores comunes: estructura inconsistente o difícil de manipular.
   - Verificar antes de seguir: poder acceder correctamente a filas y columnas.
3. Implementación de movimientos básicos: (4 abril 2026)
   - Qué hacer: desarrollar movimiento hacia la izquierda como base.
   - Debe quedar listo: desplazamiento correcto de valores sin fusión.
   - Errores comunes: no eliminar ceros correctamente.
   - Verificar antes de seguir: probar con diferentes configuraciones del tablero.
4. Implementación de fusión de baldosas: (4 abril 2026)
   - Qué hacer: implementar lógica para combinar valores iguales.
   - Debe quedar listo: fusión respetando reglas del juego.
   - Errores comunes: permitir múltiples fusiones en una sola jugada.
   - Verificar antes de seguir: probar casos consecutivos de valores iguales.
5. Movimientos en todas las direcciones: (5 abril 2026)
   - Qué hacer: extender lógica a derecha, arriba y abajo.
   - Debe quedar listo: funciones completas de movimiento.
   - Errores comunes: duplicar lógica en lugar de reutilizar funciones.
   - Verificar antes de seguir: validar consistencia en las cuatro direcciones.
6. Generación de nuevas baldosas: (5 abril 2026)
   - Qué hacer: insertar valores 2 o 4 en posiciones vacías.
   - Debe quedar listo: generación aleatoria funcional.
   - Errores comunes: seleccionar posiciones inválidas.
   - Verificar antes de seguir: comprobar que siempre aparece en espacios vacíos.
7. Condiciones de juego: (6 abril 2026)
   - Qué hacer: implementar detección de victoria y derrota.
   - Debe quedar listo: funciones gano? y perdio?.
   - Errores comunes: no detectar correctamente cuando no hay movimientos.
   - Verificar antes de seguir: probar escenarios límite.
8. Interfaz gráfica: (6–7 abril 2026)
   - Qué hacer: implementar visualización del tablero y entrada por teclado.
   - Debe quedar listo: interfaz funcional con interacción del usuario.
   - Errores comunes: problemas de actualización del tablero.
   - Verificar antes de seguir: comprobar respuesta a todas las teclas.
9. Integración del sistema: (7 abril 2026)
   - Qué hacer: unir lógica e interfaz en un solo flujo.
   - Debe quedar listo: juego completamente funcional.
   - Errores comunes: desincronización entre lógica y visualización.
   - Verificar antes de seguir: ejecutar partidas completas.
10. Pruebas y validación: (7–8 abril 2026)
    - Qué hacer: probar distintos escenarios del juego.
    - Debe quedar listo: comportamiento estable sin errores.
    - Errores comunes: probar solo casos simples.
    - Verificar antes de seguir: incluir casos extremos.

11. Documentación: (8 abril 2026)
    - Qué hacer: redactar documento técnico completo.
    - Debe quedar listo: informe con todas las secciones requeridas.
    - Errores comunes: documentación incompleta o inconsistente.
    - Verificar antes de seguir: que refleje el código implementado.
12. Preparación de entrega: (9 abril 2026)
    - Qué hacer: revisar código, documentación y repositorio.
    - Debe quedar listo: proyecto listo para entrega y defensa.
      comunes: olvidar archivos o detalles finales.
    - Verificar antes de seguir: comprobar que todo funcione correctamente.

VII. Problemas Encontrados

Durante el desarrollo se presentaron algunos problemas comunes. Inicialmente, la fusión de baldosas no funcionaba correctamente, lo cual se corrigió ajustando la lógica recursiva. También hubo dificultad al implementar movimientos en distintas direcciones, solucionado con inversión y transposición de listas. Finalmente, se complicó la selección de posiciones vacías para nuevas baldosas, lo cual se resolvió filtrando correctamente los espacios disponibles.

VIII. Conclusiones y Recomendaciones

- Cumplimiento de los requisitos: Se logró implementar una versión funcional del juego 2048 en el lenguaje Racket, utilizando programación funcional, recursividad y estructuras de datos basadas en listas, como se requería. Se implementaron los movimientos, la fusión de baldosas, y la generación de nuevas baldosas de manera eficiente.
- Desarrollo modular: El proyecto se desarrolló utilizando una clara separación de módulos, con la lógica del juego separada de la interfaz gráfica. Esto permitió un flujo de trabajo organizado y facilitó la depuración y mejoras en el futuro.
- Reutilización y transformaciones: La implementación de la lógica de los movimientos (hacia la izquierda, derecha, arriba, abajo) y la fusión de baldosas se logró de forma reutilizable mediante transformaciones del tablero, como la inversión de filas y la transposición de la matriz.
- Desafíos superados: Se encontraron y resolvieron varios problemas durante el desarrollo, como la fusión de baldosas y la selección de posiciones vacías para la generación de nuevas baldosas, que se solucionaron ajustando la lógica recursiva y filtrando correctamente las posiciones disponibles.
- Interfaz gráfica interactiva: La interfaz gráfica es funcional, permite al usuario interactuar mediante las teclas de dirección para mover las baldosas y refleja correctamente el estado del juego.

VIII-A. Recomendaciones

- Optimización en la generación de nuevas baldosas: Si bien la función de generar nuevas baldosas funciona, se recomienda optimizar el algoritmo que selecciona posiciones vacías, garantizando que no haya errores cuando el tablero esté casi lleno.
- Pruebas adicionales con escenarios límite: Es importante realizar más pruebas con escenarios límite, como tableros casi completos o movimientos complejos, para asegurar que el sistema responda correctamente en todas las situaciones.
- Mejorar la interfaz gráfica: Aunque la interfaz gráfica es funcional, se podría mejorar la visualización de los movimientos y las combinaciones, añadiendo animaciones o transiciones suaves para que la experiencia del jugador sea más interactiva y atractiva.

- Documentación y código comentado: Asegurarse de que todos los miembros del equipo comprendan a fondo el código, y que la documentación esté completa y clara, especialmente en la parte de la lógica de los movimientos y la fusión de baldosas.
- Posibles ampliaciones: Se podría considerar añadir características adicionales al juego, como la opción de deshacer movimientos, agregar un sistema de puntuación más detallado o permitir la opción de ajustar el tamaño del tablero.
- Revisión de la interacción de teclas: Asegúrese de que la interacción con las teclas sea consistente en todas las plataformas y entornos, para evitar problemas con la entrada del usuario.

IX. Repositorio

El versionamiento del proyecto se llevo a cabo en el siguiente repositorio de github: https://github.com/GldnBolt/CE2048.git.

X. Manual de usuario

X-A. *Descripción del sistema*

El presente sistema corresponde a una implementación del juego 2048 desarrollada en el lenguaje Racket. El objetivo del juego es combinar baldosas con valores iguales hasta alcanzar una baldosa con valor 2048. El sistema incluye una interfaz gráfica que permite visualizar el tablero y realizar movimientos mediante el teclado.

X-B. *Requisitos del sistema*

Para ejecutar el programa se requiere:

- Tener instalado Racket.
- Contar con un entorno compatible como DrRacket.
- Tener disponibles los archivos del proyecto, incluyendo `juego.rkt` y `logica.rkt`.

X-C. *Ejecución del programa*

Para ejecutar el sistema, se deben seguir los siguientes pasos:

1. Abrir el archivo `juego.rkt` en el entorno DrRacket.
2. Ejecutar el programa utilizando el botón de *Run*.
3. Se desplegará una ventana con el tablero del juego.

X-D. *Uso del sistema*

El usuario interactúa con el juego utilizando las teclas de dirección del teclado:

- Flecha izquierda: mueve las baldosas hacia la izquierda.
- Flecha derecha: mueve las baldosas hacia la derecha.
- Flecha arriba: mueve las baldosas hacia arriba.
- Flecha abajo: mueve las baldosas hacia abajo.

En cada movimiento:

- Las baldosas se desplazan en la dirección indicada.
- Las baldosas con el mismo valor se combinan en una sola.
- Se genera una nueva baldosa (2 o 4) en una posición vacía.

X-E. *Condiciones del juego*

El sistema contempla dos condiciones principales:

- **Victoria**: se alcanza cuando una baldosa con valor 2048 aparece en el tablero.
- **Derrota**: ocurre cuando no existen movimientos posibles en el tablero.

X-F. Ejemplo de uso

Al iniciar el juego, el tablero contiene dos baldosas con valor 2 en posiciones aleatorias. El usuario puede presionar la flecha izquierda para combinar valores. Por ejemplo, si existen dos baldosas con valor 2 en la misma fila, estas se combinarán en una baldosa con valor 4.

X-G. Consideraciones

- Cada movimiento válido genera una nueva baldosa.
- Una baldosa solo puede fusionarse una vez por jugada.
- El tablero se actualiza automáticamente después de cada acción.
- Si no hay cambios en el tablero tras una acción, no se genera una nueva baldosa.

REFERENCIAS

[1] Racket Documentation. *Racket Official Documentation*. [Enlace: https://docs.racket-lang.org/]. Accedido: abril, 2026.
[2] Wikipedia. *2048 (game)*. [Enlace: https://en.wikipedia.org/wiki/2048_(video_game)]. Accedido: abril, 2026.
[3] Otros recursos utilizados durante el desarrollo del proyecto.